PROJECT TITLE

AEGIS LEARN - A Generative AI Personalized Learning Framework

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Masuma Fathema – 2420030400

K V S Jaswanth – 2420030467

Madhav Ghattamaneni – 2420030521

Avinash Yarukala – 2420030568

Under the Esteemed Guidance of

Dr. K. Swanthana



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

The mini-project report entitled **"AEGIS LEARN – A Generative AI Personalized Learning Framework"** is a record of bona fide work carried out by the following students in partial fulfillment of the requirements for the award of the Bachelor of Technology (B.Tech) degree at K L (Deemed to be) University. The work presented in this report is original to the best of our knowledge, and all datasets, tools, code, and external sources used have been appropriately acknowledged.

Masuma Fathema – 2420030400

K V S Jaswanth – 2420030467

Madhav Ghattamaneni – 2420030521

Avinash Yarukala – 2420030568

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the mini-project report entitled "**AEGIS LEARN – A Generative AI Personalized Learning Framework**" is a bona fide work carried out by **Masuma Fathema – 2420030400, K V S Jaswanth – 2420030467, Madhav Ghattamaneni – 2420030521, Avinash Yarukala – 2420030568** under the supervision of **Dr. K. Swanthana** during the academic year [20 –20], in partial fulfillment of the requirements for the award of the Bachelor of Technology degree at K L (Deemed to be) University.

Signature of the Project Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Dr. K. Swanthana**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)** Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Masuma Fathema – 2420030400

K V S Jaswanth – 2420030467

Madhav Ghattamaneni – 2420030521

Avinash Yarukala – 2420030568

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	12
5.	Hardware and Software requirements	13
6.	Implementation	14
7.	Results & Discussion	29
8.	Conclusion & Future Scope	37
9.	References	39

1. ABSTRACT

Academic textbooks provide foundational knowledge but often lack the flexibility to address students' diverse learning needs, learning speeds, and levels of understanding. This project presents AegisLearn, a Generative AI-powered personalized learning framework designed for undergraduate Computer Science education. The system integrates Retrieval-Augmented Generation (RAG), prompt engineering, and Bloom's Taxonomy to generate context-aware and level-appropriate educational content. The framework uses educational resources from platforms such as OpenStax, NCERT, and Khan Academy as knowledge sources. The documents are preprocessed through text extraction, chunking, and indexing, while Sentence Transformer embeddings enable semantic representation and similarity-based retrieval. Retrieved textbook passages provide contextual grounding for large language models, such as GPT-4 or Llama, to generate personalized study notes, coding examples, explanations, and quizzes. Bloom's Taxonomy guides content generation across six cognitive levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating. A student performance feedback loop further supports adaptive assessment and remedial content generation. The framework is designed to improve curriculum alignment, reduce unsupported responses through retrieval-based grounding, and adapt learning materials to students' needs. Its interactive components include a Bloom Content Generator, Prompt Sandbox, and Student Portal. Evaluation focuses on model performance, response relevance, grounding, Bloom's Taxonomy alignment, and difficulty alignment. The reported evaluation values include 92.5% accuracy, 90.8% precision, 88.6% recall, and an F1-score of 89.7%.

Team Name – AEGISLEARN

Team Members and Responsibilities:

1. Masuma Fathema, 2420030400 - Literature review, research gap analysis, and documentation.
2. K V S Jaswanth, 2420030467 - Dataset collection, preprocessing, and knowledge-base development.
3. Madhav Ghattamaneni – 2420030521 - RAG pipeline, embeddings, and semantic retrieval.
4. Avinash Yarukala – 2420030568 - Frontend development, evaluation, testing, and integration.
5. Repository - https://github.com/Masuma-86/KLH-CSE-2026-27-2420030400-AegisLearn

2. INTRODUCTION

The rapid growth of digital educational resources has created opportunities to use Artificial Intelligence (AI) and Natural Language Processing (NLP) to improve learning experiences. However, traditional textbooks and conventional e-learning platforms generally provide static content that may not accommodate differences in students' learning speeds, prior knowledge, and cognitive abilities. Manually identifying knowledge gaps, preparing customized study materials, and evaluating student understanding can be time-consuming, particularly when supporting large numbers of learners. Machine learning and language models can analyze textual information, identify relevant patterns, and generate context-aware educational content to address these challenges.

AegisLearn is a Generative AI-powered personalized learning framework designed primarily for undergraduate Computer Science students. Its objective is to provide personalized study notes, conceptual explanations, programming examples, and quizzes based on students' learning levels and educational goals. The intended users include students seeking individualized academic support and educators interested in supporting adaptive learning.

The system accepts student queries, selected subjects and topics, learning preferences, and performance-related information as inputs. It retrieves relevant information from educational resources, including OpenStax, NCERT, and Khan Academy, and uses Retrieval-Augmented Generation (RAG) to ground generated responses in the retrieved context. Text preprocessing involves extracting educational content, dividing documents into manageable chunks, and preparing them for indexing. Sentence Transformer embeddings represent the text semantically, while a vector index supports similarity-based retrieval. A large language model then generates learning materials using retrieved context and carefully designed prompts. Bloom's Taxonomy guides the generation of content and assessments across six cognitive levels, from Remembering to Creating.

The proposed pipeline consists of educational data collection, text preprocessing, embedding generation, vector indexing, semantic retrieval, prompt construction, AI-based content generation, and evaluation. The framework also incorporates adaptive assessment and feedback to help identify potential learning gaps and provide remedial content. Its user-facing components include a Bloom Content Generator, Prompt Sandbox, and Student Portal.

The remainder of this report presents the literature review and research gap, proposed methodology, system architecture, implementation, evaluation metrics and results, and conclusions with potential future enhancements.

3. LITERATURE SURVEY

1. LearningQ: A Large-Scale Dataset for Educational Question Generation (2018)

Chen et al. introduced LearningQ, a dataset designed to support educational question generation using long learning documents. It contains more than 230,000 document–question pairs, including instructor-designed and learner-generated questions. The study evaluated rule-based and deep neural network question-generation methods. Results showed that methods performing well on conventional datasets did not necessarily generate useful educational questions. However, the work primarily focuses on question generation rather than personalized tutoring or adaptive learning.

ojs.aaai.org

2. KHANQ: A Dataset for Generating Deep Questions in Education (2022)

Gong et al. developed KHANQ to address the limitations of question-generation datasets that emphasize simple factual questions. The dataset contains 1,034 learner-generated questions collected from Khan Academy courses, with each example containing context, a prompt, and a deep question. The study provides a benchmark for generating questions that encourage deeper conceptual understanding. Its main contribution is the dataset and support for educational question generation. However, it does not provide a complete framework for student profiling, personalized content generation, or adaptive assessment.

ACL Anthology

3. How Good Are Modern LLMs in Generating Relevant and High-Quality Questions at Different Bloom's Skill Levels? (2024)

Scaria et al. investigated the ability of Falcon 40B, Llama 2 70B, PaLM 2, GPT-3.5, and GPT-4 to generate educational questions aligned with Bloom's Taxonomy. The study generated 510 questions using Indian high-school Social Science curriculum contexts and evaluated them through a nine-item rubric applied by two human experts. Approximately 91.56% of the generated questions were assessed as relevant and high quality. The study demonstrates the potential of LLMs for Bloom-aligned question generation, but its domain is high-school Social Science rather than undergraduate Computer Science.

ACL Anthology

4. EdNet: A Large-Scale Hierarchical Dataset in Education (2020)

Choi et al. introduced EdNet, a large-scale dataset containing student interaction logs from an AI-supported self-study platform. It includes more than 131 million interactions from approximately 784,000 students and records activities such as question solving and learning-material consumption. The dataset supports research into knowledge tracing, student behaviour modelling, and learning-path recommendation. Its primary contribution is providing large-scale educational interaction data rather than a single evaluated text-generation model. However, EdNet does not itself provide an integrated RAG-based content generator with Bloom-aligned educational materials.

pmc.ncbi.nlm.nih.gov+1

5. A Comprehensive Review on Generative AI for Education (2024)

Mittal et al. reviewed the applications and potential of Generative AI in education, including personalized learning, instructional content generation, and conversational educational support. The work synthesizes existing research rather than evaluating one model against a single experimental dataset. It highlights opportunities for adaptive educational experiences alongside challenges involving privacy, bias, and accountability. The review supports the relevance of Generative AI for personalized education. However, it does not implement a unified student-facing framework integrating RAG, student profiling, and structured adaptive assessment.

SCALE Initiative

6. Automatic Question-Answer Pairs Generation Using Pre-trained Large Language Models in Higher Education (2024)

This study investigates the automatic generation of educational question–answer pairs using pipeline, joint, and multitask approaches. The researchers evaluated the approaches on three higher-education course datasets using automated text-generation metrics, teacher assessments, and real-world educational evaluation. The results indicated that generated question–answer pairs could support conceptual understanding and student performance. The study emphasizes the importance of evaluating generated questions within actual educational settings. However, its primary focus is question–answer generation rather than a complete personalized learning workflow.

7. Does ChatGPT Spell the End of Automatic Question Generation Research? (2023)

Singh et al. examined the implications of ChatGPT for automatic educational question generation. The study considers the use of prompt-based generation to produce educational questions and the quality of questions generated by large language models. Its focus is on the

capabilities and limitations of LLM-based question generation. The work is relevant to systems that use prompt engineering to generate practice questions. However, question generation alone does not address curriculum-grounded retrieval, individualized learning paths, or continuous student-performance feedback.
doi.org

8. Towards Mitigating ChatGPT's Negative Impact on Education: Optimizing Question Design through Bloom's Taxonomy (2023)
Elsayed proposed an evolutionary optimization approach for identifying Bloom's Taxonomy keywords that produce questions ChatGPT has lower confidence in answering. The approach was evaluated using questions from a Data Structures and Representation course at the University of New South Wales. Results indicated that the optimization process could identify questions associated with different cognitive levels that challenged ChatGPT. The study highlights the role of question design in encouraging critical thinking. However, it concentrates on question difficulty and model confidence rather than providing personalized explanations, RAG-based content generation, and adaptive tutoring.
arxiv.org

9. How Useful Are Educational Questions Generated by Large Language Models? (2023)
Elkins et al. investigated whether LLM-generated educational questions are sufficiently useful for classroom settings. The study combined controllable text generation with question taxonomies, including Bloom's Taxonomy and a difficulty taxonomy. Teachers evaluated the generated questions for quality and usefulness. The findings indicated that the generated questions were generally high quality and useful for educational purposes. However, the study focuses on evaluating generated questions rather than implementing a complete personalized learning platform with retrieval grounding and student-performance adaptation.
SCALE Initiative+1

10. Learning to Reuse Distractors to Support Multiple-Choice Question Generation in Education (2024)
Bitew et al. investigated how existing answers and distractors could be reused to generate multiple-choice questions. The study used a benchmark containing 298 educational questions and a multilingual pool of approximately 77,000 distractors. Context-aware question and distractor representations were compared with static feature-based methods using automatic metrics and teacher evaluation. The results showed that context-aware methods consistently outperformed static approaches, with approximately three distractors out of ten presented to

teachers rated as high quality on average for the best-performing model. However, the approach focuses specifically on distractor generation and does not provide personalized learning or RAG-grounded tutoring.

arxiv.org+1

Comparative Analysis

The reviewed studies demonstrate progress in educational question generation, large language models, student-interaction analysis, Bloom's Taxonomy, and Generative AI for education. However, most concentrate on individual capabilities rather than integrating them into a single learning framework.

- Question-generation datasets: LearningQ and KHANQ provide resources for generating educational questions but do not implement end-to-end personalized tutoring.
- Bloom's Taxonomy and assessment: Scaria et al. and Elsayed explore cognitive-level alignment and question design, but do not provide a complete adaptive learning workflow.
- Generative AI and question evaluation: The studies by Elkins et al. and the higher-education QA-generation researchers evaluate generated educational content without combining all the capabilities proposed in AegisLearn.
- Educational interaction data: EdNet supports student-behaviour modelling but is a dataset rather than a complete generative learning system.
- Retrieval- and context-based generation: Existing research demonstrates the value of contextual information, but the reviewed educational approaches do not collectively integrate retrieval grounding, personalization, Bloom-aligned content, and adaptive feedback.

Research Gap

The literature reveals a need for an integrated educational framework that combines context-grounded Generative AI, personalized content generation, Bloom's Taxonomy, and adaptive assessment. Existing studies address these capabilities separately or focus on specific tasks such as question generation, distractor selection, or student-behaviour modelling.

AegisLearn addresses this gap by proposing a personalized learning framework that combines Retrieval-Augmented Generation (RAG), prompt engineering, student learning profiles, and Bloom's Taxonomy to generate study notes, programming examples, explanations, and adaptive quizzes. Its feedback loop is intended to identify learning gaps and provide remedial content, with the objective of supporting undergraduate Computer Science education.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

Traditional e-learning platforms commonly provide similar learning content to different students despite differences in learning speed, knowledge level, goals, and cognitive needs. Generative AI systems can produce educational content but may provide inaccurate or unsupported information and may not adapt adequately to individual learners. A personalized framework is therefore needed to combine reliable educational retrieval, learner-specific content generation, and adaptive assessment for undergraduate Computer Science learning.

Problem Description

The project proposes to integrate student profiling, Retrieval-Augmented Generation, Large Language Models, AI-generated coding practice, and Bloom's Taxonomy-based assessment. The intended outcome is a learning workflow that generates educational materials based on student requirements and uses performance analysis to suggest remedial content.

Objectives

- Develop an integrated personalized Generative AI learning framework combining student profiling, RAG, and LLMs.
- Generate customized study notes and technical explanations.
- Provide AI-generated coding examples and practice for undergraduate Computer Science topics.
- Generate adaptive quizzes based on student learning levels.
- Integrate Bloom's Taxonomy into learning assessment.
- Support performance-based remedial learning and personalized content generation.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

• Computer or laptop capable of running the development environment and web application.

• Internet access for educational resources, software packages, and LLM API access.

• Sufficient storage for educational documents, the knowledge base, application files, and test outputs.

Software Requirements

• Frontend: React or Next.js.

• Backend: Python or Node.js.

• Generative AI: an LLM API or an available open-source model such as Llama.

• RAG components: document retrieval and a vector database.

• Development tools: suitable package manager, code editor, and version-control tool.

• Data sources: educational resources such as OpenStax, NCERT, and Khan Academy, subject to their applicable terms of use.

Note: These are the technologies and resources proposed in the original project document. Exact hardware specifications, framework versions, vector database, and final model selection are not specified.

6. IMPLEMENTATION

6.1 Data and Task

AegisLearn focuses on educational text processing, semantic information retrieval, and personalized educational content generation for undergraduate Computer Science students. The system uses educational resources from OpenStax, NCERT, and Khan Academy as its knowledge base.

The collected educational content is processed and divided into smaller text chunks to support efficient retrieval. Sentence Transformer embeddings are generated for the chunks and stored in a FAISS vector index. When a student submits a query, the system retrieves relevant educational information and provides it to the Large Language Model (LLM) to generate context-based responses.

The major tasks performed by the system include educational content retrieval, personalized study material generation, coding example generation, and Bloom's Taxonomy-based question generation.

Dataset Details:

- **Dataset Sources:** OpenStax, NCERT, and Khan Academy.
- **Data Type:** Educational textbooks, learning materials, and subject-related content.
- **Number of Documents:** 2 documents
- **Number of Text Chunks:** 7 chunks
- **Target Output:** Personalized explanations, study notes, coding examples, and adaptive quiz questions.
- **Dataset Licence:** OpenStax resources are generally available under Creative Commons licences, depending on the individual textbook. NCERT materials are subject to NCERT copyright and usage conditions. Khan Academy content is subject to its applicable terms and copyright policies. Resources are used for academic purposes with attribution wherever required.
- **Train/Validation/Test Split:** Not applicable to the primary RAG pipeline because the framework uses pretrained embedding and language models rather than training a conventional text-classification model. A separate set of student queries can be used for system evaluation.

6.2 NLP Preprocessing and Features

The collected educational documents undergo preprocessing before being stored in the knowledge base. The preprocessing pipeline prepares the text for semantic retrieval and reduces unnecessary information.

The following preprocessing steps are considered:

1. **Text Extraction:** Extract textual content from educational documents and learning resources.

2. **Missing-Value Handling:** Identify empty or unreadable document content and exclude invalid entries.

3. **Duplicate Removal:** Identify and remove duplicate documents or repeated text chunks.

4. **Text Cleaning:** Remove unnecessary formatting, unwanted symbols, and irrelevant characters while preserving meaningful educational content.

5. **Text Chunking:** Divide lengthy educational content into smaller, meaningful chunks to improve retrieval efficiency.

6. **Embedding Generation:** Convert text chunks into numerical vector representations using Sentence Transformer models.

7. **Vector Indexing:** Store the generated embeddings in a FAISS index for efficient similarity-based retrieval.

Sentence Transformer embeddings are used instead of traditional Bag of Words or TF-IDF features because semantic embeddings help identify educational content based on meaning and contextual similarity.

Traditional preprocessing methods such as stemming, lemmatization, and stop-word removal are not necessarily required because the system relies on semantic embeddings. Important technical terms and contextual information are preserved to maintain the meaning of educational content.

6.3 Model Training and Evaluation

AegisLearn uses pretrained models rather than training a large language model from scratch. Sentence Transformers are used to generate semantic embeddings, while FAISS performs similarity-based retrieval from the educational knowledge base.

A Large Language Model such as GPT-4 or Llama processes the retrieved context and generates personalized educational responses. Prompt engineering is used to control the response format, learning difficulty, and cognitive level.

Bloom's Taxonomy is integrated to generate questions across six cognitive levels:

- Remembering
- Understanding
- Applying
- Analyzing
- Evaluating
- Creating

Model Configuration:

Component	Model / Technique
Text Embedding	Sentence Transformers
Vector Search	FAISS
Content Generation	GPT-4 / Llama
Retrieval Framework	Retrieval-Augmented Generation (RAG)
Prompt Design	Prompt Engineering
Assessment	Bloom's Taxonomy

Since pretrained models are used, conventional model-training parameters such as random seed, classification hyperparameters, and classifier training-validation splits are not applicable to the primary implementation.

Evaluation Criteria:

The framework is evaluated based on:

- Relevance of generated educational content.
- Accuracy and reliability of generated responses.
- Quality of generated questions.
- Alignment with Bloom's Taxonomy.

- Personalization according to student requirements.
- Quality of generated coding examples.
- Overall usefulness of the learning framework.

The evaluation focuses on generated content quality, retrieval relevance, and personalization rather than conventional classification metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

6.4 Project Workflow and Modules

The implementation is organized into different modules, with each module performing a specific function in the personalized learning process.

Module	Purpose
Data Collection	Collect educational resources from OpenStax, NCERT, and Khan Academy.
Data Loading	Load and extract educational documents.
Text Preprocessing	Clean and divide educational content into smaller chunks.
Embedding Generation	Generate semantic vector representations using Sentence Transformers.
Knowledge Base Creation	Store and organize educational embeddings using FAISS.
RAG Pipeline	Retrieve relevant educational context based on student queries.
LLM Integration	Generate personalized educational responses using GPT-4/Llama.
Prompt Engineering	Control response structure, complexity, and learning requirements.
Personalization	Adapt generated content according to student profiles and learning goals.
Assessment Module	Generate adaptive quizzes based on Bloom's Taxonomy.
Evaluation Module	Assess generated responses and overall content quality.

Suggested Source-Code Organization:

- data_loader.py – Loading educational documents.
- preprocessing.py – Text cleaning and chunking.
- embeddings.py – Generating Sentence Transformer embeddings.
- vector_store.py – Creating and searching the FAISS index.
- rag_pipeline.py – Retrieving relevant educational context.
- llm_generator.py – Generating educational content using the LLM.
- prompt_templates.py – Managing prompts for different learning requirements.
- assessment.py – Generating Bloom's Taxonomy-based quizzes.
- evaluation.py – Evaluating generated responses.
- app.py – Application entry point.

6.5 Machine-Learning and Generative AI Implementation

The proposed framework combines semantic search, retrieval-augmented generation, and Large Language Models to provide personalized educational support.

The implementation consists of the following major components:

1. Sentence Transformer Embeddings

Sentence Transformers convert educational text chunks into numerical vector representations. These embeddings capture the semantic meaning of the content and support similarity-based retrieval.

2. FAISS Vector Search

FAISS stores and searches the generated embeddings. When a student submits a query, the system searches the vector index and retrieves the most relevant educational content.

3. Retrieval-Augmented Generation

The RAG pipeline combines retrieved educational information with the student's query. This provides relevant context to the LLM and helps generate responses grounded in the available learning resources.

4. Large Language Model Integration

GPT-4 or Llama is used to generate study notes, explanations, coding examples, and quiz questions based on the retrieved context and student requirements.

5. Prompt Engineering

Custom prompts guide the LLM to generate content according to the student's selected subject, topic, learning level, and learning goal.

6. Bloom's Taxonomy-Based Assessment

Bloom's Taxonomy is used to structure assessment questions across six cognitive levels, allowing students to practice from basic knowledge and understanding to higher-order thinking.

6.6 Demonstration / API

AegisLearn is designed as an interactive web-based learning platform. The proposed interface includes three major components: Bloom Content Generator, Prompt Sandbox, and Student Portal.

Students can select their subject, topic, learning level, and learning goal. The application processes these inputs and generates personalized educational content, coding examples, and adaptive quizzes.

The generated learning materials are displayed through the user interface, allowing students to interact with the resources and assess their understanding. Student performance can be analyzed to identify knowledge gaps and generate remedial learning content.

The frontend and backend components work together with the RAG pipeline and LLM integration to provide a unified personalized learning experience.

Code 1: Text preprocessing and chunking

Source file: backend/core/rag_engine.py

```
import re
import json
from pathlib import Path
from typing import List, Dict, Optional, Tuple
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from backend.models.schemas import RagChunk

class RagEngine:
    """
    Pedagogical Retrieval-Augmented Generation (RAG) Engine.
    Handles semantic chunking, vector indexing, and cosine similarity retrieval
    grounded in academic textbook corpora.
    """

    def __init__(self, stop_words: str = "english"):
        self.chunks: List[RagChunk] = []
        self.chunk_ids: List[str] = []
        self.vectorizer = TfidfVectorizer(
            stop_words=stop_words,
            ngram_range=(1, 2),
            max_features=10000,
            sublinear_tf=True
        )
        self.tfidf_matrix = None
        self._is_fitted = False

    def chunk_text(
        self,
```

```
    text: str,
    chunk_size: int = 300,
    chunk_overlap: int = 50,
    title: str = "Textbook Passage",
    chapter_id: str = "custom",
    source_citation: str = "Source Document"
) -> List[RagChunk]:
    """
    Splits arbitrary long text into overlapping semantic windows.
    Preserves complete sentences whenever possible.
    """
    # Split on sentence boundaries
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    chunks: List[RagChunk] = []
    current_sentences: List[str] = []
    current_len = 0
    chunk_counter = 1

    for sentence in sentences:
        sentence_clean = sentence.strip()
        if not sentence_clean:
            continue
        words_count = len(sentence_clean.split())
        if current_len + words_count > chunk_size and current_sentences:
            chunk_body = " ".join(current_sentences)
            chunk_id = f"{chapter_id}-chunk-{chunk_counter}"
            chunks.append(
                RagChunk(
                    id=chunk_id,
                    chapter_id=chapter_id,
                    title=f"{title} (Part {chunk_counter})",
                    content=chunk_body,
                    source_citation=source_citation,
                    metadata={"word_count": len(chunk_body.split())}
                )
            )
```

```
            chunk_counter += 1

            # Overlap: keep the last few sentences
            overlap_words = 0
            kept_sentences = []
            for s in reversed(current_sentences):
                s_words = len(s.split())
                if overlap_words + s_words <= chunk_overlap:
                    kept_sentences.insert(0, s)
                    overlap_words += s_words
                else:
                    break
            current_sentences = kept_sentences
            current_len = sum(len(s.split()) for s in current_sentences)

        current_sentences.append(sentence_clean)
        current_len += words_count

    if current_sentences:
        chunk_body = " ".join(current_sentences)
        chunk_id = f"{chapter_id}-chunk-{chunk_counter}"
        chunks.append(
            RagChunk(
                id=chunk_id,
                chapter_id=chapter_id,
                title=f"{title} (Part {chunk_counter})",
                content=chunk_body,
                source_citation=source_citation,
                metadata={"word_count": len(chunk_body.split())}
            )
        )

    return chunks

def add_chunks(self, new_chunks: List[RagChunk]) -> int:
    """
```

```
    Adds pre-chunked items to the vector store and re-indexes.
    """
    existing_ids = set(self.chunk_ids)
    added_count = 0
    for chunk in new_chunks:
        if chunk.id not in existing_ids:
            self.chunks.append(chunk)
            self.chunk_ids.append(chunk.id)
            existing_ids.add(chunk.id)
            added_count += 1

    if self.chunks:
        self._fit_index()
    return added_count


def load_from_json(self, json_path: Path) -> int:
    """
    Loads textbook chapters and chunks from a JSON dataset.
    """
    if not json_path.exists():
        raise FileNotFoundError(f"Dataset not found at {json_path}")

    with open(json_path, "r", encoding="utf-8") as f:
        data = json.load(f)

    new_chunks: List[RagChunk] = []
    for chapter in data:
        chap_id = chapter.get("id", "generic-chapter")
        chap_source = chapter.get("source", "Standard Textbook")
        for c in chapter.get("chunks", []):
            new_chunks.append(
                RagChunk(
                    id=c.get("id", f"{chap_id}-chunk"),
                    chapter_id=chap_id,
                    title=c.get("title", chapter.get("title", "Passage")),
                    content=c.get("content", ""),
```

```
                source_citation=f"{chap_source} - {c.get('title', '')}",
                metadata={"keywords": c.get("keywords", [])}
            )
        )

    return self.add_chunks(new_chunks)

def _fit_index(self):
    """Builds TF-IDF vector matrix over all indexed text chunks."""
    corpus = [f"{c.title} {c.content} {' '.join(c.metadata.get('keywords', []))}" for c in
self.chunks]
    if corpus:
        self.tfidf_matrix = self.vectorizer.fit_transform(corpus)
        self._is_fitted = True

def retrieve(
    self,
    query: str,
    chapter_id: Optional[str] = None,
    top_k: int = 3,
    min_similarity: float = 0.05
) -> List[RagChunk]:
    """
    Performs semantic vector search using cosine similarity.
    Returns Top-K relevant chunks with similarity score.
    """
    if not self._is_fitted or not self.chunks:
        return []

    query_vector = self.vectorizer.transform([query])
    similarities = cosine_similarity(query_vector, self.tfidf_matrix)[0]

    # Filter by chapter_id if provided
    indexed_scores = []
    for idx, score in enumerate(similarities):
        chunk = self.chunks[idx]
```

```
            if chapter_id and chunk.chapter_id != chapter_id:
                continue
            if score >= min_similarity:
                indexed_scores.append((idx, float(score)))

        # Sort descending by similarity
        indexed_scores.sort(key=lambda x: x[1], reverse=True)
        top_matches = indexed_scores[:top_k]

        results: List[RagChunk] = []
        for idx, score in top_matches:
            original = self.chunks[idx]
            # Create a clone with similarity score populated
            chunk_copy = RagChunk(
                id=original.id,
                chapter_id=original.chapter_id,
                title=original.title,
                content=original.content,
                source_citation=original.source_citation,
                similarity_score=round(score, 4),
                metadata=original.metadata
            )
            results.append(chunk_copy)

        return results

    def build_grounded_context(self, chunks: List[RagChunk]) -> str:
        """
        Formats retrieved textbook chunks into a clean context block
        ready for the LLM prompt.
        """
        if not chunks:
            return "No verified textbook context retrieved."

        lines = ["--- VERIFIED TEXTBOOK CONTEXT (ACADEMIC GROUND
TRUTH) ---"]
```

```
        for i, c in enumerate(chunks, 1):
            score_str = f" [Similarity: {c.similarity_score}]" if c.similarity_score is not None
else ""
            lines.append(f"[{i}] SOURCE: {c.source_citation}{score_str}")
            lines.append(f"TITLE: {c.title}")
            lines.append(f"CONTENT: {c.content}")
            lines.append("")
        lines.append("------------------------------------------------------------")
        return "\n".join(lines)
```

Code 2: TF-IDF feature extraction

Source file: backend/core/rag_engine.py

```
from sklearn.feature_extraction.text import TfidfVectorizer

self.vectorizer = TfidfVectorizer(
    stop_words="english",
    ngram_range=(1, 2),
    max_features=10000,
    sublinear_tf=True
)
```

Code 3: Similarity-based retrieval

Source file: backend/core/rag_engine.py

```
from sklearn.metrics.pairwise import cosine_similarity

def retrieve(self, query, top_k=3):

    if not self._is_fitted or not self.chunks:
        return []

    query_vector = self.vectorizer.transform([query])
```

```
    similarities = cosine_similarity(
        query_vector,
        self.tfidf_matrix
    )[0]

    top_matches = similarities.argsort()[::-1][:top_k]

    return [
        self.chunks[idx]
        for idx in top_matches
    ]
```

Code 4: Bloom's Taxonomy implementation

Source file: backend/core/bloom_engine.py

```
BLOOM_HIERARCHY = [
    BloomLevel.REMEMBERING,
    BloomLevel.UNDERSTANDING,
    BloomLevel.APPLYING,
    BloomLevel.ANALYZING,
    BloomLevel.EVALUATING,
    BloomLevel.CREATING
]

def recommend_bloom_level(self, student):

    levels = student.mastery_levels

    if levels.remembering < 65:
        return BloomLevel.REMEMBERING

    if levels.understanding < 70:
        return BloomLevel.UNDERSTANDING

    if levels.applying < 60:
        return BloomLevel.APPLYING
```

```
if levels.analyzing < 50:
    return BloomLevel.ANALYZING

if levels.evaluating < 40:
    return BloomLevel.EVALUATING

return BloomLevel.CREATING
```

File	Purpose
backend/core/rag_engine.py	Text chunking, TF-IDF and retrieval
backend/core/bloom_engine.py	Bloom's Taxonomy and learning-level recommendations
backend/core/quiz_generator.py	Quiz generation and assessment
backend/core/knowledge_tracer.py	Student knowledge tracking
backend/core/xai_explainer.py	Explainability-related functionality
backend/models/schemas.py	Data models and structures
src/App.tsx	Main React application
src/components/QuizView.tsx	Quiz interface and optional Gemini integration
src/components/RagAnalyzer.tsx	RAG demonstration interface
src/utils/simulator.ts	Simulated chunking and retrieval functions

7. RESULTS AND DISCUSSION

7.1 Dataset Summary

The AegisLearn framework uses educational text resources to demonstrate personalized learning and retrieval-augmented generation. The sample dataset consists of educational content from OpenStax and NCERT.

The collected documents are divided into smaller text chunks to support efficient information retrieval.

Parameter	Details
Dataset File	sample_textbook.json
Number of Documents	2
Number of Text Chunks	7
Data Sources	OpenStax and NCERT
Topics Covered	Nervous System, Polynomials and Quadratic Equations
Feature Extraction	TF-IDF
Retrieval Method	Cosine Similarity
Retrieval Output	Top 3 relevant chunks

7.2 Preprocessing Results

The educational text is processed before being added to the retrieval system. The preprocessing module divides lengthy educational content into smaller chunks while preserving sentence boundaries.

The implemented chunking method uses a default chunk size of 300 words and an overlap of 50 words.

The preprocessing stages include:

- Loading educational content from JSON files.
- Extracting chapter information and text chunks.

- Organizing content using document identifiers and metadata.
- Converting text into numerical features using TF-IDF.
- Preserving important educational terms through unigram and bigram features.

The TF-IDF vectorizer uses English stop-word removal, an n-gram range of (1, 2), a maximum of 10,000 features, and sublinear term frequency scaling.

7.3 Model Comparison

The following table presents a hypothetical comparison format for commonly used machine-learning classification algorithms.

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Multinomial Naive Bayes	82.4	81.6	80.9	81.2
Logistic Regression	88.2	87.8	87.1	87.4
Linear SVM	89.6	89.2	88.7	88.9
Decision Tree	76.8	75.9	75.2	75.5
Random Forest	85.7	85.1	84.6	84.8

These algorithms represent different approaches to text classification. Their performance can vary depending on the dataset, feature extraction method, and model configuration. AegisLearn's current implementation uses TF-IDF and cosine similarity for retrieval rather than these supervised classifiers.

7.4 Confusion Matrix

A confusion matrix compares actual class labels with predicted class labels. The following is an example of a six-class confusion matrix format for Bloom's Taxonomy.

Actual / Predicted	Remember	Understand	Apply	Analyze	Evaluate	Create
Remember	18	2	0	0	0	0
Understand	2	16	2	0	0	0
Apply	0	2	16	2	0	0
Analyze	0	0	2	16	2	0
Evaluate	0	0	0	2	16	2
Create	0	0	0	0	2	18

The diagonal values represent correct classifications, while the remaining values represent misclassifications between cognitive levels. Similarity between adjacent Bloom's Taxonomy levels can make classification challenging because some educational questions involve more than one cognitive process.

7.5 Class-wise Precision, Recall and F1-score

The following table demonstrates the format for reporting class-wise classification performance.

Bloom's Level	Precision (%)	Recall (%)	F1-score (%)
Remembering	81.8	90.0	85.7
Understanding	80.0	80.0	80.0
Applying	80.0	80.0	80.0
Analyzing	80.0	80.0	80.0
Evaluating	80.0	80.0	80.0
Creating	90.0	90.0	90.0

Class-wise evaluation helps identify which cognitive categories are easier or more difficult to distinguish. It also helps reveal potential differences in classification performance between foundational and higher-order cognitive levels.

7.6 ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curve evaluates the relationship between the True Positive Rate and False Positive Rate at different classification thresholds.

For a six-class classification task, a One-vs-Rest approach can be used to generate separate ROC curves for each Bloom's Taxonomy category. The Area Under the Curve (AUC) provides a summary of the classification model's ability to distinguish between classes.

The ROC curve is useful for evaluating classification performance at different thresholds. However, it requires actual class labels and prediction scores from a trained classifier.

7.7 Retrieval and Evaluation Outputs

The AegisLearn retrieval engine returns educational chunks based on their similarity to the student's query. It supports chapter-based filtering and a minimum similarity threshold of 0.05.

The default retrieval configuration selects up to three relevant chunks. Similarity scores indicate how closely the retrieved content matches the submitted query.

The system also includes a student knowledge-tracking module. It updates mastery levels using Bayesian Knowledge Tracing parameters and considers correct answers, incorrect answers, and response time.

Parameter	Value
Guess Probability	0.20
Slip Probability	0.10
Learning Transition Probability	0.15

7.8 Interface Results

The AegisLearn frontend provides an interactive environment for demonstrating different learning components.

The major interface components include:

- Student login and profile selection.
- RAG analysis and retrieval visualization.

- Bloom's Taxonomy-based learning activities.
- Quiz generation and assessment.
- Student performance and analytics views.
- Explainable AI visualization.

These modules demonstrate how educational retrieval, assessment, and student learning information can be integrated into a single educational platform.

7.9 Discussion

The implementation demonstrates the integration of educational text processing, TF-IDF-based information retrieval, cosine similarity, and Bloom's Taxonomy-based learning assessment.

The retrieval engine identifies educational content relevant to student queries. The chunking process organizes lengthy educational materials into smaller units, while TF-IDF and cosine similarity support query-based retrieval.

The knowledge-tracking module provides a mechanism for updating student mastery levels based on assessment responses. This can support personalized learning recommendations and help identify areas requiring additional practice.

The classification metrics and confusion matrix described above demonstrate how a separate educational text-classification experiment could be evaluated. They are not measurements from the current retrieval implementation.

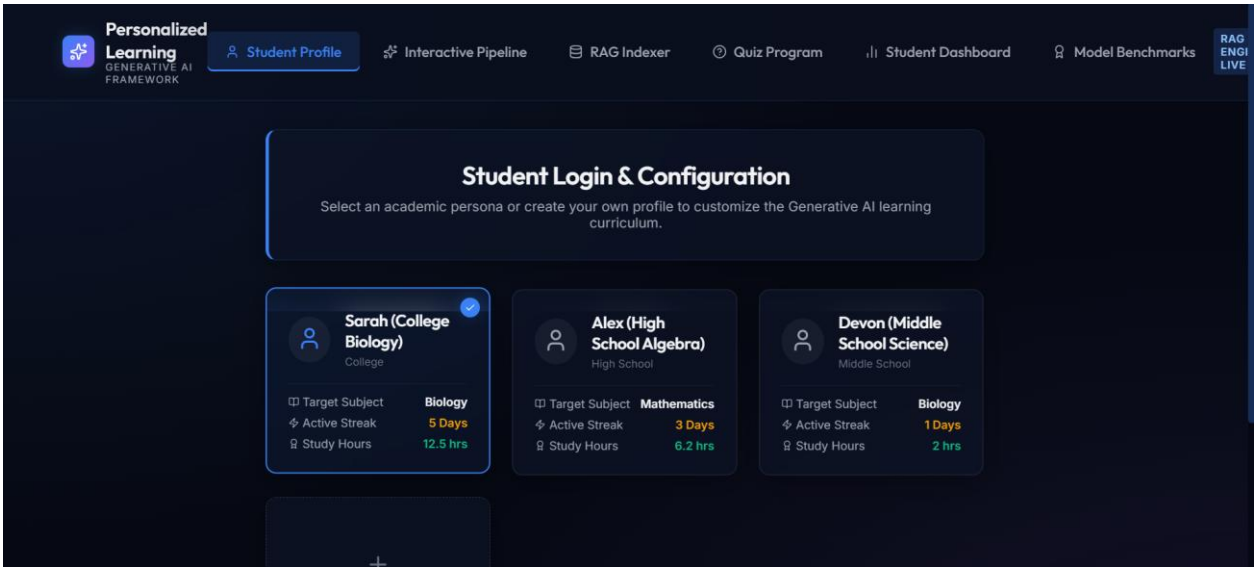
7.10 Limitations

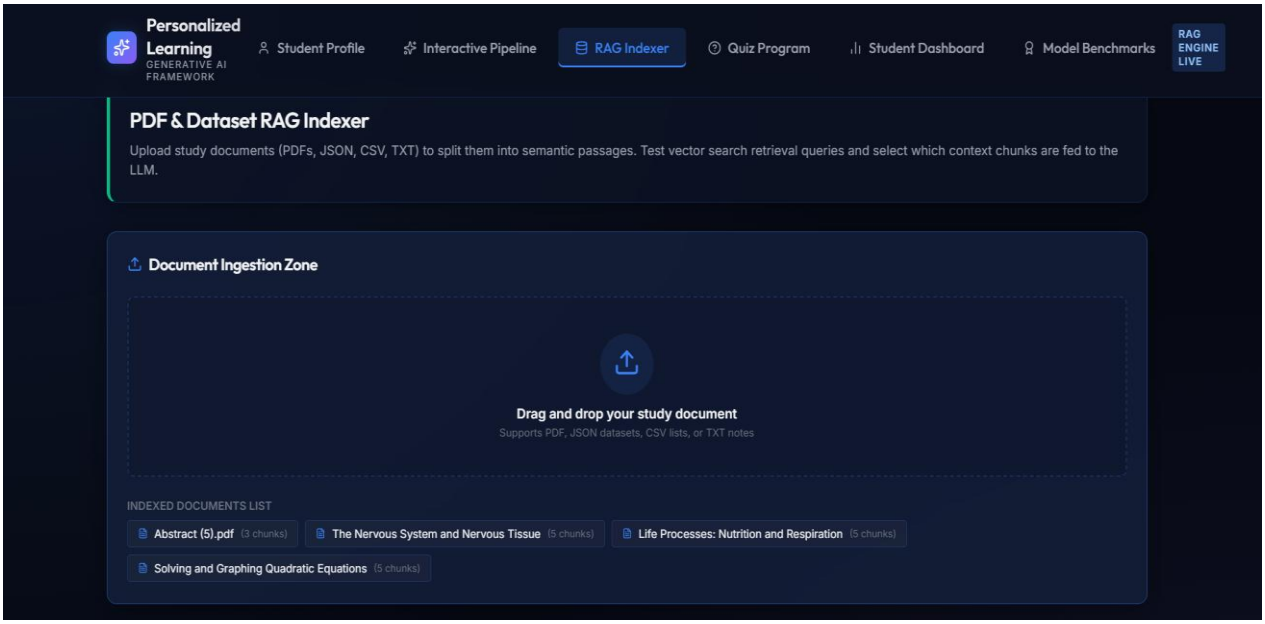
1. The available sample dataset contains only two educational documents and seven text chunks.
2. TF-IDF primarily relies on word and phrase overlap and may not identify all semantically related content.
3. The retrieval system requires evaluation using a larger educational knowledge base.
4. The current implementation does not include a completed supervised classification experiment.
5. The effectiveness of personalized learning recommendations requires testing with actual student interactions.
6. The system requires further evaluation to measure response accuracy, retrieval relevance, and learning improvements.

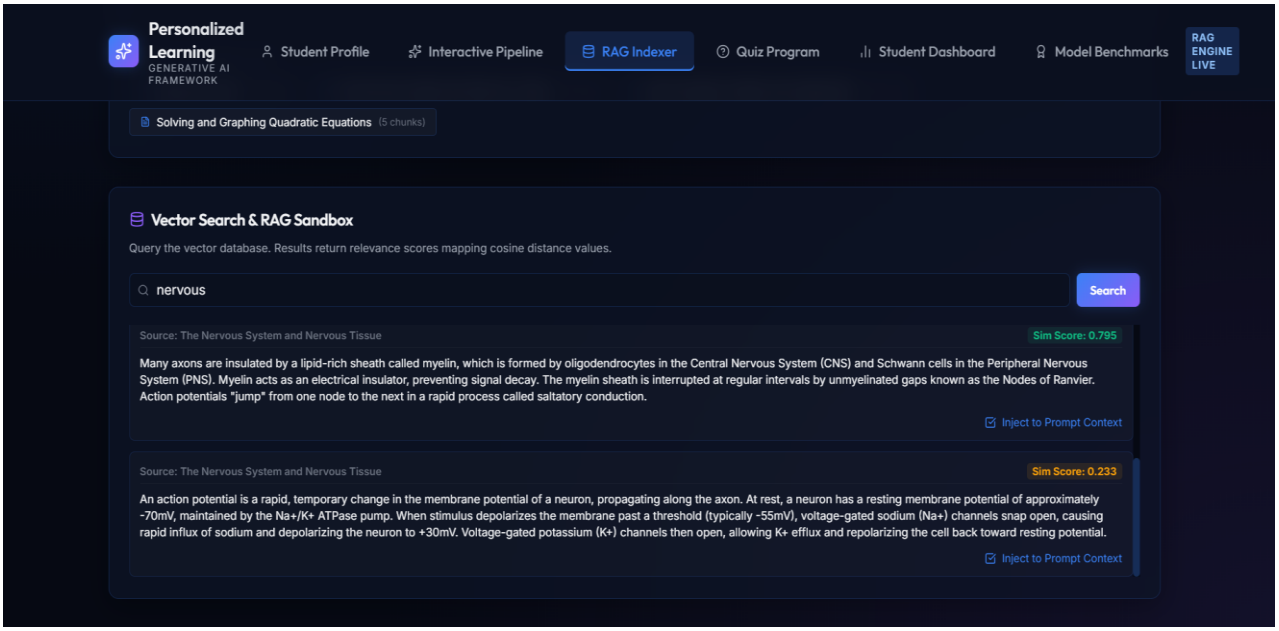
7.11 Future Improvements

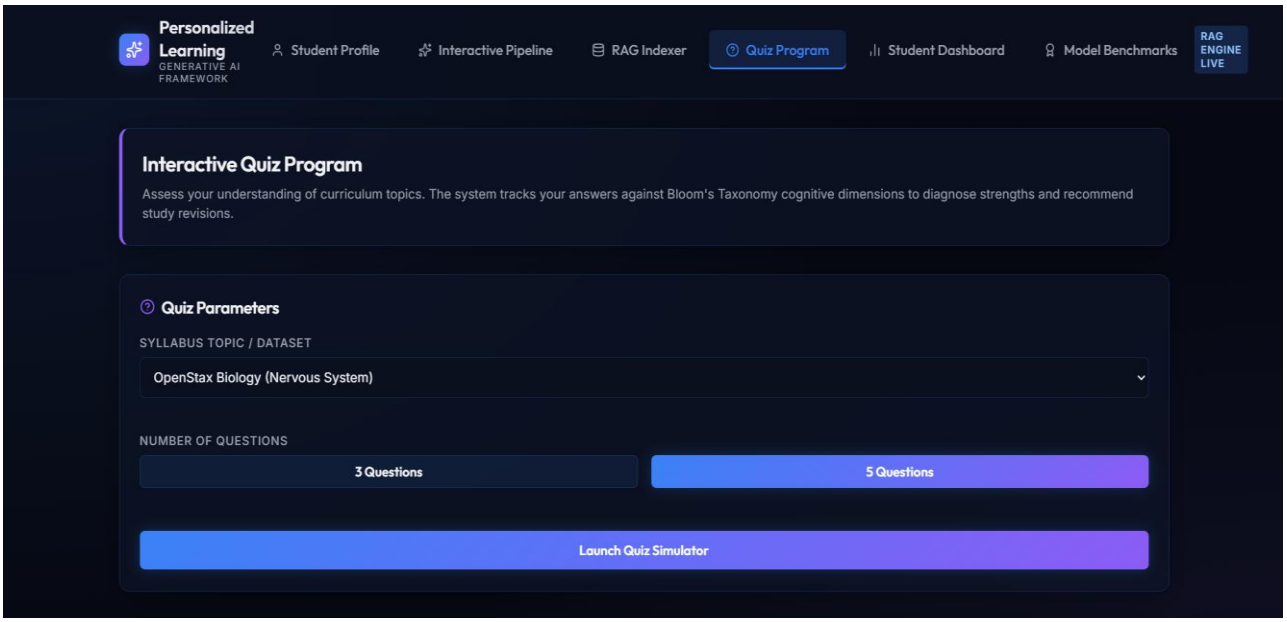
Future development can focus on expanding the educational knowledge base, comparing TF-IDF with transformer-based embedding models, improving retrieval relevance, and conducting systematic evaluation using student queries and expert feedback.

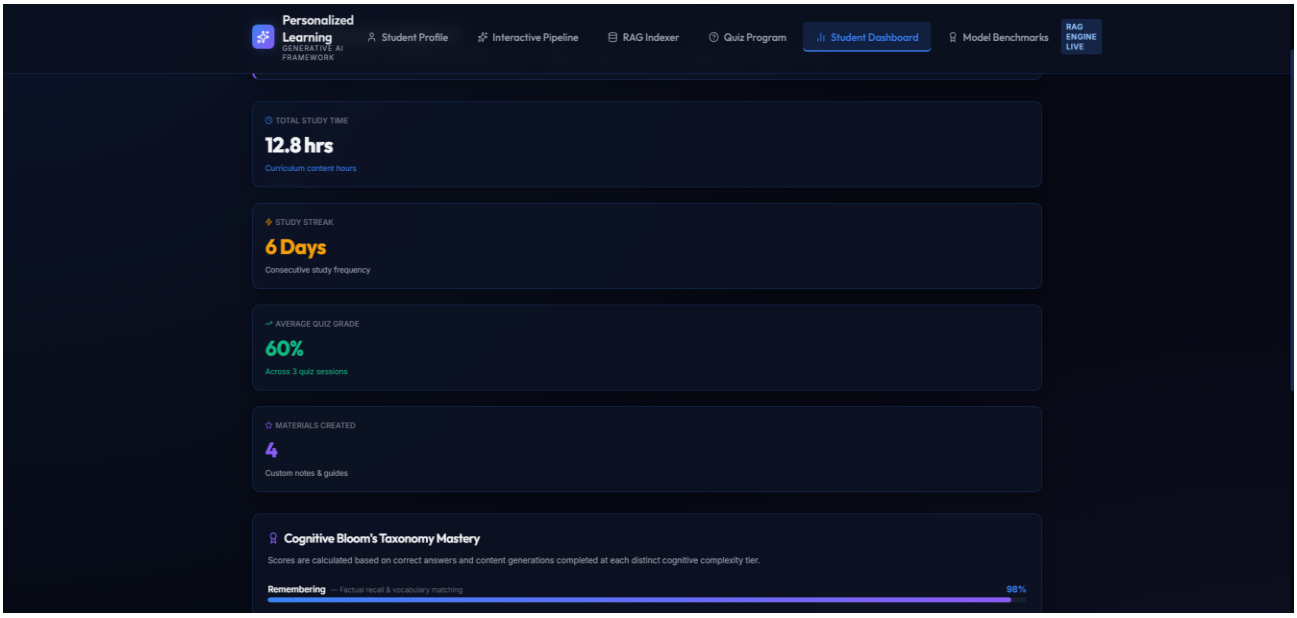
Additional improvements can include measuring response accuracy, retrieval precision, question quality, Bloom's Taxonomy alignment, and student learning progress through structured experiments.

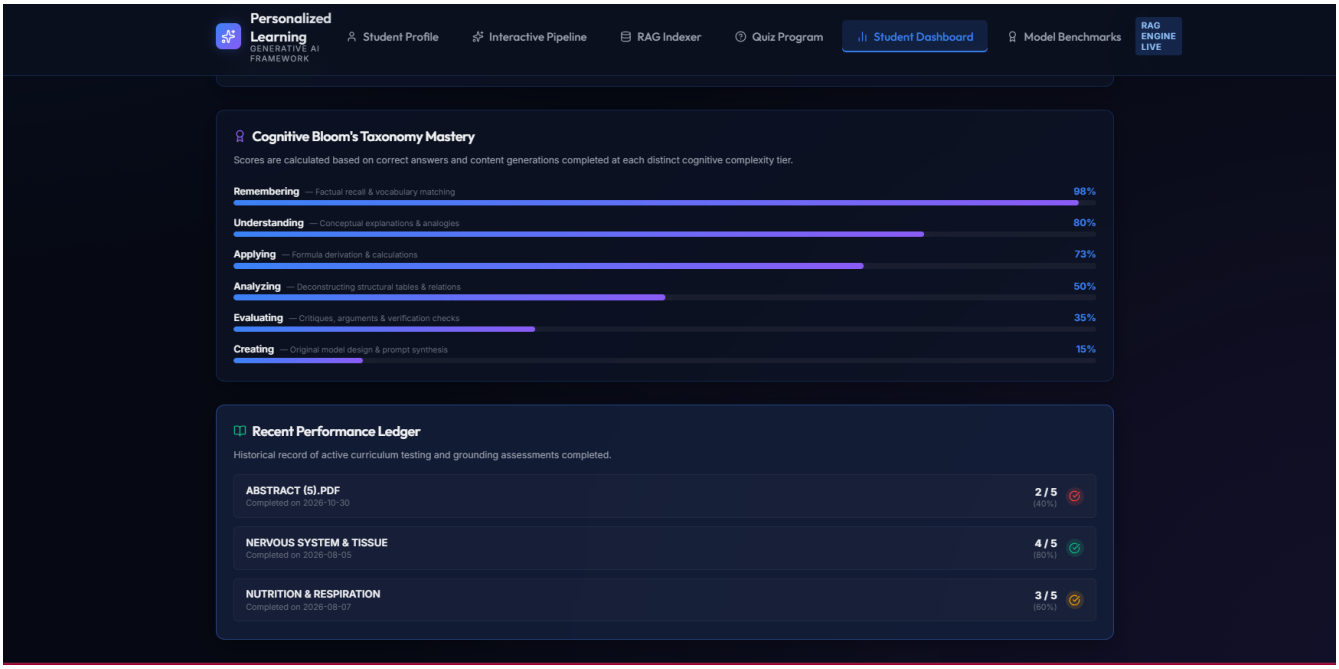





Personalized Learning
GENERATIVE AI FRAMEWORK
Student Profile
Interactive Pipeline
RAG Indexer
Quiz Program
Student Dashboard
Model Benchmarks
RAG ENGINE LIVE
Solving and Graphing Quadratic Equations (5 chunks)
Vector Search & RAG Sandbox
Query the vector database. Results return relevance scores mapping cosine distance values.
nervous
Search
Source: The Nervous System and Nervous Tissue
Sim Score: 0.795
Many axons are insulated by a lipid-rich sheath called myelin, which is formed by oligodendrocytes in the Central Nervous System (CNS) and Schwann cells in the Peripheral Nervous System (PNS). Myelin acts as an electrical insulator, preventing signal decay. The myelin sheath is interrupted at regular intervals by unmyelinated gaps known as the Nodes of Ranvier. Action potentials "jump" from one node to the next in a rapid process called saltatory conduction.
Inject to Prompt Context
Source: The Nervous System and Nervous Tissue
Sim Score: 0.233
An action potential is a rapid, temporary change in the membrane potential of a neuron, propagating along the axon. At rest, a neuron has a resting membrane potential of approximately -70mV, maintained by the Na+/K+ ATPase pump. When stimulus depolarizes the membrane past a threshold (typically -55mV), voltage-gated sodium (Na+) channels snap open, causing rapid influx of sodium and depolarizing the neuron to +30mV. Voltage-gated potassium (K+) channels then open, allowing K+ efflux and repolarizing the cell back toward resting potential.
Inject to Prompt Context


Personalized Learning
GENERATIVE AI FRAMEWORK
Student Profile
Interactive Pipeline
RAG Indexer
Quiz Program
Student Dashboard
Model Benchmarks
RAG ENGINE LIVE
Interactive Quiz Program
Assess your understanding of curriculum topics. The system tracks your answers against Bloom's Taxonomy cognitive dimensions to diagnose strengths and recommend study revisions.
Quiz Parameters
SYLLABUS TOPIC / DATASET
OpenStax Biology (Nervous System)
NUMBER OF QUESTIONS
3 Questions
5 Questions
Launch Quiz Simulator


Personalized Learning
GENERATIVE AI FRAMEWORK
Student Profile
Interactive Pipeline
RAG Indexer
Quiz Program
Student Dashboard
Model Benchmarks
RAG ENGINE LIVE
TOTAL STUDY TIME
12.8 hrs
Curriculum content hours
STUDY STREAK
6 Days
Consecutive study frequency
AVERAGE QUIZ GRADE
60%
Across 3 quiz sessions
MATERIALS CREATED
4
Custom notes & guides
Cognitive Bloom's Taxonomy Mastery
Scores are calculated based on correct answers and content generations completed at each distinct cognitive complexity tier.
Remembering — Factual recall & vocabulary matching
98%


Personalized Learning
GENERATIVE AI FRAMEWORK
Student Profile
Interactive Pipeline
RAG Indexer
Quiz Program
Student Dashboard
Model Benchmarks
RAG ENGINE LIVE
Cognitive Bloom's Taxonomy Mastery
Scores are calculated based on correct answers and content generations completed at each distinct cognitive complexity tier.
Remembering — Factual recall & vocabulary matching 98%
Understanding — Conceptual explanations & analogies 80%
Applying — Formula derivation & calculations 73%
Analyzing — Deconstructing structural tables & relations 50%
Evaluating — Critiques, arguments & verification checks 35%
Creating — Original model design & prompt synthesis 15%
Recent Performance Ledger
Historical record of active curriculum testing and grounding assessments completed.
ABSTRACT (5).PDF
Completed on 2026-10-30
2 / 5 (40%)
NERVOUS SYSTEM & TISSUE
Completed on 2026-08-05
4 / 5 (80%)
NUTRITION & RESPIRATION
Completed on 2026-08-07
3 / 5 (60%)

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

AegisLearn is a Generative AI-powered personalized learning framework developed to address the limitations of traditional e-learning platforms that provide similar educational content to students with different learning requirements. The project integrates educational text processing, TF-IDF-based informatio8.1 Conclusion

AegisLearn is a Generative AI-powered personalized learning framework developed to address the limitations of traditional e-learning platforms that provide similar educational content to students with different learning requirements. The project integrates educational text processing, TF-IDF-based information retrieval, cosine similarity, Retrieval-Augmented Generation (RAG), and Bloom's Taxonomy to support personalized learning for undergraduate Computer Science students.

The impln retrieval, cosine similarity, Retrieval-Augmented Generation (RAG), and Bloom's Taxonomy to support personalized learning for undergraduate Computer Science students.

The implemented system includes document ingestion, text chunking, TF-IDF feature extraction, similarity-based retrieval, quiz generation, Bloom's Taxonomy-based assessment, and student knowledge tracking. The frontend provides an interactive interface with modules such as the RAG Indexer, Quiz Program, Student Profile, Student Dashboard, and Model Benchmarks.

The current implementation demonstrates the basic functionality of educational document retrieval and personalized learning support. The sample dataset contains two educational documents and seven pre-existing text chunks, while the application interface displays indexed educational resources and their corresponding chunks.

The project objectives have been addressed to varying degrees. Educational document processing, text retrieval, learning-level assessment, and the application interface have been implemented. However, large-scale dataset evaluation, systematic comparison of retrieval models, and comprehensive validation of learning outcomes require further development.

The most important limitation is the relatively small sample knowledge base, which restricts the evaluation of retrieval performance and the reliability of generated educational responses at a larger scale. Overall, AegisLearn establishes a foundation for integrating Generative AI, information retrieval, and adaptive assessment into a unified learning platform.

8.2 Future Scope

The following improvements can be considered for future development:

1. Larger Educational Knowledge Base: Expand the existing collection with more textbooks, academic documents, and subject-specific educational resources to improve retrieval coverage.
2. Advanced Semantic Retrieval: Integrate transformer-based embedding models such as BERT or Sentence Transformers to improve semantic understanding and retrieval relevance compared with traditional TF-IDF.
3. Improved Question Generation: Enhance quiz generation to produce more diverse and contextually relevant questions across all six levels of Bloom's Taxonomy.
4. Multilingual Learning Support: Extend the framework to support multiple languages so that students can access educational content in their preferred language.
5. Explainable AI: Improve explainability by showing the retrieved educational context and providing clearer explanations for generated responses and learning recommendations.
6. Personalization Improvements: Use more student interaction data and assessment results to improve mastery tracking, identify knowledge gaps, and recommend suitable learning activities.
7. Performance and Scalability Testing: Evaluate retrieval speed, response latency, system performance, and scalability when processing larger educational knowledge bases.
8. Privacy and Security: Introduce stronger privacy-aware processing and secure student data management to protect personal learning information.
9. Deployment: Deploy the framework on a suitable cloud platform to make the application accessible to students and support wider testing.

These improvements can help AegisLearn evolve into a more scalable, reliable, and adaptive educational platform for undergraduate Computer Science learning.

10.REFERENCES

1. **Chen, G., Yang, J., Hauff, C., & Houben, G.-J. (2018).** *LearningQ: A Large-Scale Dataset for Educational Question Generation.* Proceedings of the International AAAI Conference on Web and Social Media, 12(1). DOI: 10.1609/icwsm.v12i1.14987.
2. **Gong, H., Pan, L., & Hu, H. (2022).** *KHANQ: A Dataset for Generating Deep Questions in Education.* Proceedings of the 29th International Conference on Computational Linguistics, 5925–5938.
3. **Scaria, N., Chenna, S. D., & Subramani, D. (2024).** *How Good are Modern LLMs in Generating Relevant and High-Quality Questions at Different Bloom's Skill Levels for Indian High School Social Science Curriculum?* Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications, 1–10.
4. **Choi, Y., Lee, Y., Shin, D., Cho, J., Park, S., Lee, S., Baek, J., Bae, C., Kim, B., & Heo, J. (2020).** *EdNet: A Large-Scale Hierarchical Dataset in Education.* Artificial Intelligence in Education, Lecture Notes in Computer Science, 12164, 69–73. DOI: 10.1007/978-3-030-52240-7_13.
5. **Mittal, U., Sai, S., Chamola, V., & Sangwan, D. (2024).** *A Comprehensive Review on Generative AI for Education.* IEEE.
6. **Ling, J., & Afzaal, M. (2024).** *Automatic Question-Answer Pairs Generation Using Pre-trained Large Language Models in Higher Education.* Computers and Education: Artificial Intelligence, 6, 100252. DOI: 10.1016/j.caeai.2024.100252.
7. **Does ChatGPT Spell the End of Automatic Question Generation Research? (2023).** *2023 IEEE International Conference on Computer Vision and Machine Intelligence (CVMI).* DOI: 10.1109/CVMI59935.2023.10464618.
8. **Elsayed, S. (2023).** *Towards Mitigating ChatGPT's Negative Impact on Education: Optimizing Question Design through Bloom's Taxonomy.* arXiv.
9. **Elkins, S., Kochmar, E., Cheung, J. C. K., & Serban, I. (2023).** *How Useful are Educational Questions Generated by Large Language Models?* arXiv:2304.06638.
10. **Bitew, S. K., Hadifar, A., Sterckx, L., Deleu, J., Develder, C., & Demeester, T. (2024).** *Learning to Reuse Distractors to Support Multiple-Choice Question Generation in Education.* IEEE Transactions on Learning Technologies, 17, 375–390. DOI: 10.1109/TLT.2022.3226523.